

Temas Tratados en el Trabajo Práctico 8

- Aprendizaje estadístico.
- Evolución de la verosimilitud de una hipótesis en función de observaciones.
- Aprendizaje no supervisado. Algoritmo K-means.
- Aprendizaje supervisado. Algoritmo Knn.
- Aprendizaje por refuerzo. Algoritmo Q-Learning.

Ejercicios Teóricos

1. Un fabricante de tornillos vende cajas que contienen 1000 tornillos de cabeza redonda con tres tipos de recubrimiento electrolítico (cincado, cobre y níquel). Cada caja se rellena con diferentes proporciones que pueden variar de la siguiente manera:

a: Todos los tornillos están recubiertos de níquel. 15 de cada 100 cajas se llenan de esta manera.

b: El 70% de los tornillos están recubiertos de níquel, el 20% de cobre, el resto está cincado. 15 de cada 100 cajas se llenan de esta manera.

c: El 50% de los tornillos están recubiertos de níquel, el 25% de cobre y el resto está cincado. 50 de cada 100 cajas se llenan de esta manera.

d: El 20 % de los tornillos están recubiertos de níquel, el 50% de cobre y el resto está cincado. 10 de cada 100 cajas se llenan de esta manera.

e: Todos los tornillos están recubiertos de cobre. 10 de cada 100 cajas se llenan de esta manera.

1.1 ¿Cuál es la distribución a priori sobre las hipótesis?

La distribución a priori es:

$P(h_1): 0.15$

$P(h_2): 0.15$

$P(h_3): 0.50$

$P(h_4): 0.10$

$P(h_5): 0.10$

Considerando que los 10 primeros tornillos que se extraen de una caja de muestra son de cobre:

1.2 Calcule la probabilidad de cada hipótesis dado que los 10 primeros tornillos fueron de cobre.

```

from PIL import Image
import matplotlib.pyplot as plt

# Abrir la imagen desde la carpeta del proyecto
img = Image.open("tp8_1_1.jpeg")

# Mostrar la imagen con matplotlib
plt.figure(figsize=(img.width/80, img.height/80)) # Ajustá el divisor
para el tamaño
plt.imshow(img)
plt.axis("off")
plt.show()

```

		P priori											
hi	P(cobre hi)	P(hi)	P(hi d1)	P(hi d2)	P(hi d3)	P(hi d4)	P(hi d5)	P(hi d6)	P(hi d7)	P(hi d8)	P(hi d9)	P(hi d10)	
a	0.000	0.150	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
b	0.200	0.150	0.098	0.037	0.010	0.002	0.000	0.000	0.000	0.000	0.000	0.000	0.000
c	0.250	0.500	0.410	0.193	0.064	0.018	0.005	0.001	0.000	0.000	0.000	0.000	0.000
d	0.500	0.100	0.164	0.154	0.103	0.058	0.030	0.015	0.008	0.004	0.002	0.001	0.001
e	1.000	0.100	0.328	0.616	0.823	0.922	0.965	0.983	0.992	0.996	0.998	0.999	0.999
		P(D)	0.305	0.532	0.749	0.892	0.956	0.981	0.991	0.996	0.998	0.999	
		alfa	3.279	1.880	1.335	1.121	1.046	1.019	1.009	1.004	1.002	1.001	
											chequeo=	1.000	

1.3 Grafique la evolución de la verosimilitud de cada hipótesis en función del número de tornillos extraídos de la caja.

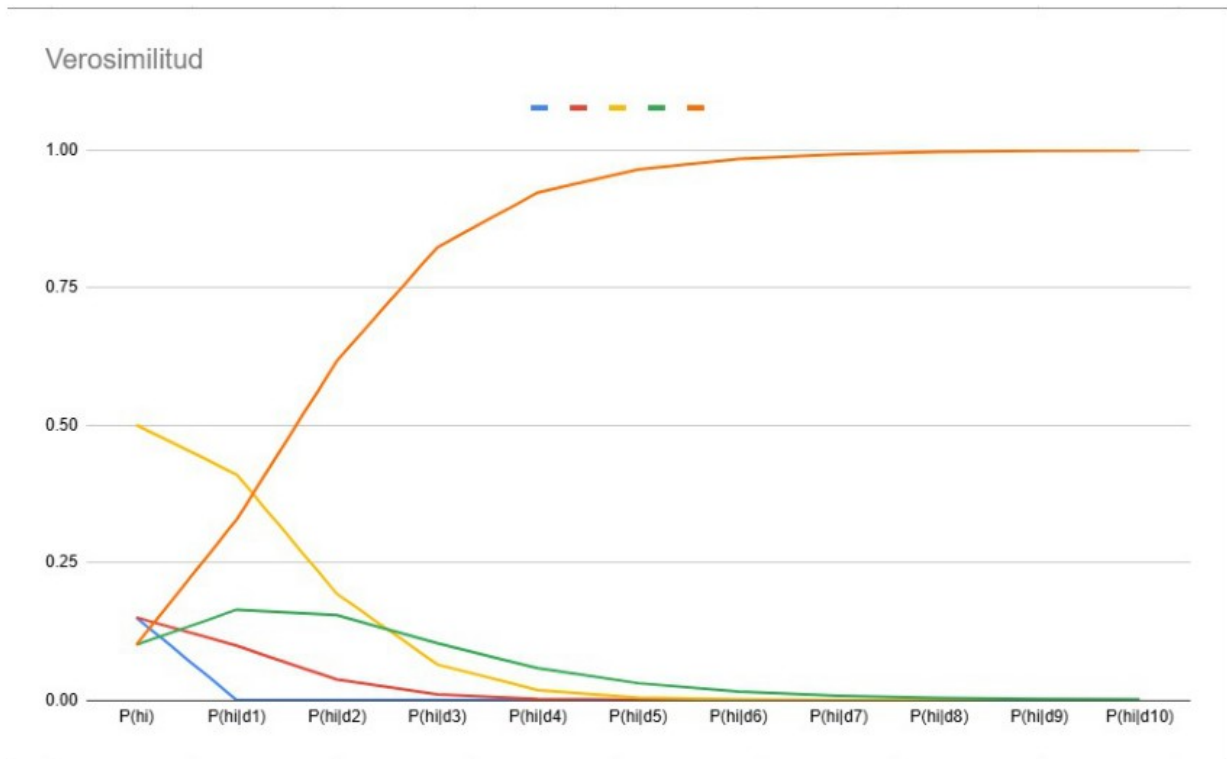
```

from PIL import Image
import matplotlib.pyplot as plt

# Abrir la imagen desde la carpeta del proyecto
img = Image.open("tp8_1_2.jpeg")

# Mostrar la imagen con matplotlib
plt.figure(figsize=(img.width/80, img.height/80)) # Ajustá el divisor
para el tamaño
plt.imshow(img)
plt.axis("off")
plt.show()

```



1.4 Para cada hipótesis, ¿cuál es la probabilidad de que el cuarto tornillo extraído sea de cobre?

$$P(h_1|\text{cobre}): (0.00)^4 = 0.0000$$

$$P(h_1|\text{cobre}): (0.20)^4 = 0.0016$$

$$P(h_1|\text{cobre}): (0.25)^4 = 0.0039$$

$$P(h_1|\text{cobre}): (0.50)^4 = 0.0625$$

$$P(h_1|\text{cobre}): (1.00)^4 = 0.1$$

1. ¿Qué diferencia principal existe entre un algoritmo supervisado y un algoritmo no supervisado?

En un algoritmo supervisado, el algoritmo aprende a partir de un conjunto de datos de entrenamiento que ha sido previamente etiquetado, el objetivo de este es aprender una función o "mapeo" que pueda predecir la salida correcta para nuevos datos de entrada que no ha visto antes. En cambio un algoritmo no supervisado, el algoritmo trabaja con datos que no están etiquetados, el principal objetivo es explorar los datos y encontrar patrones, estructuras o agrupaciones inherentes.

Ejercicios de implementación

1. Genere un conjunto de 23 puntos que contengan coordenadas x y y aleatorias con valores contenidos en el intervalo $[0, 5]$ y grafique los puntos obtenidos en un gráfico.

3.1 Implemente un algoritmo K-means que clasifique 20 de los puntos en 2 grupos y grafique el resultado asignando un color a cada uno.

```
import numpy as np
import matplotlib.pyplot as plt

def ejecutar_kmeans():

    np.random.seed(44)
    datos = np.random.rand(23, 2) * 5
    puntos_a_clasificar = datos[:20]
    puntos_restantes = datos[20:]

    k = 2
    max_iter = 30

    centroides = np.random.rand(k, 2) * 5
    print("--- Inicialización ---")
    print(f"Centroides iniciales aleatorios:\n{centroides}\n")

    for i in range(max_iter):
        # Se guarda una copia de los centroides antes de la
        # actualización.
        centroides_anteriores = centroides.copy()

        #Asignamos
        asignaciones = np.zeros(puntos_a_clasificar.shape[0],
dtype=int)
        for j, punto in enumerate(puntos_a_clasificar):
            distancias = [np.linalg.norm(punto - centroide) for
centroide in centroides]
            asignaciones[j] = np.argmin(distancias)

        #Actualizamos
        for cluster_idx in range(k):
            puntos_en_cluster = puntos_a_clasificar[asignaciones ==
cluster_idx]
            if len(puntos_en_cluster) > 0:
                centroides[cluster_idx] =
puntos_en_cluster.mean(axis=0)

        #Comprobamos
        if np.array_equal(centroides, centroides_anteriores):
            print(f"Algoritmo convergido en la iteración {i+1} por
estabilidad de centroides.")
            break

        if i == max_iter - 1:
            print(f"Algoritmo alcanzó el máximo de {max_iter}
iteraciones sin converger.")
```

```

print("\n--- Resultados Finales ---")
print(f"Centroides finales:\n{centroides}")

#Graficamos
plt.figure(figsize=(10, 8))
colores = ['blue', 'red']

for cluster_idx in range(k):
    puntos_del_cluster = puntos_a_clasificar[asignaciones ==
cluster_idx]
    plt.scatter(puntos_del_cluster[:, 0], puntos_del_cluster[:,
1],
                c=colores[cluster_idx], label=f'Grupo {cluster_idx
+ 1}', alpha=0.7)

    plt.scatter(puntos_restantes[:, 0], puntos_restantes[:, 1],
                c='black', marker='s', label='Puntos no clasificados')

    plt.scatter(centroides[:, 0], centroides[:, 1], c='yellow',
marker='X', s=200,
                edgecolors='black', label='Centroides Finales')

plt.title('Clasificación con K-Means (Convergencia por
Centroides)')
plt.xlabel('Coordenada X')
plt.ylabel('Coordenada Y')
plt.legend()
plt.grid(True)
plt.show()

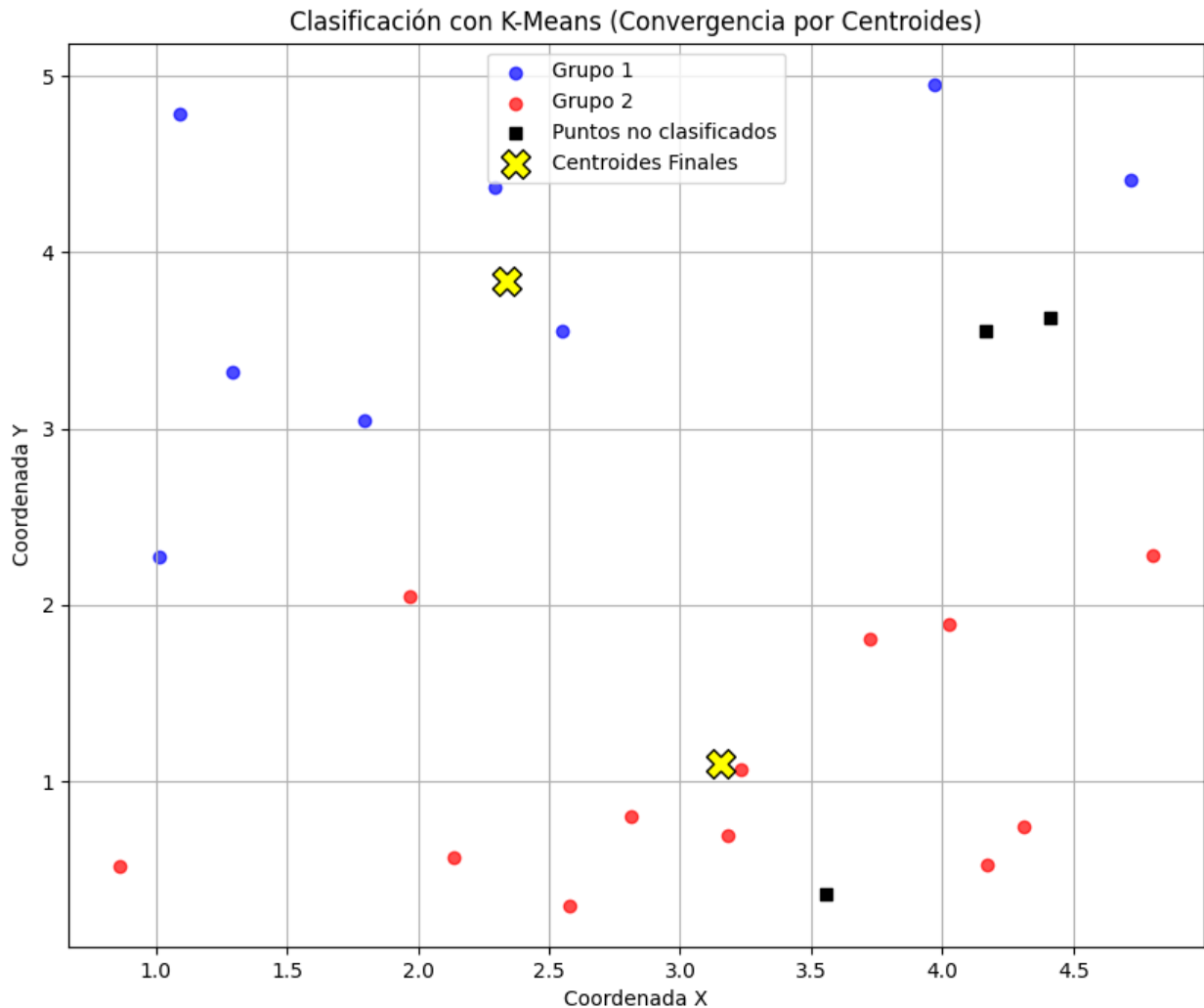
# Ejecutar la función
ejecutar_kmeans()

--- Inicialización ---
Centroides iniciales aleatorios:
[[3.48714959 4.65051402]
 [4.40836374 0.47485992]]

Algoritmo convergido en la iteración 2 por estabilidad de centroides.

--- Resultados Finales ---
Centroides finales:
[[2.34070608 3.83963062]
 [3.15164848 1.10240252]]

```



3.2 Tome los tres puntos restantes y clasifíquelos en los grupos obtenidos anteriormente usando el algoritmo Knn. Utilice distintos valores de K y anote lo que observa con esta elección.

```
import numpy as np
import matplotlib.pyplot as plt

def ejecutar_kmeans_y_knn():
    np.random.seed(44)
    datos = np.random.rand(23, 2) * 5
    puntos_a_clasificar = datos[:20]
    puntos_restantes = datos[20:]

    k = 2
    max_iter = 30

    centroides = np.random.rand(k, 2) * 5
```

```

for i in range(max_iter):
    centroides_anteriores = centroides.copy()

    # Asignación
    asignaciones = np.zeros(puntos_a_clasificar.shape[0],
dtype=int)
    for j, punto in enumerate(puntos_a_clasificar):
        distancias = [np.linalg.norm(punto - centroide) for
centroide in centroides]
        asignaciones[j] = np.argmin(distancias)

    # Actualización
    for cluster_idx in range(k):
        puntos_en_cluster = puntos_a_clasificar[asignaciones ==
cluster_idx]
        if len(puntos_en_cluster) > 0:
            centroides[cluster_idx] =
puntos_en_cluster.mean(axis=0)

    # Verificación de Convergencia
    if np.array_equal(centroides, centroides_anteriores):
        print(f"Algoritmo K-Means convergido en la iteración
{i+1}.")
        break
    if i == max_iter - 1:
        print(f"K-Means alcanzó el máximo de {max_iter}
iteraciones.")

    k_vecinos = 3 #Numero de vecinos
    predicciones_knn = []

    print(f"\n--- Clasificando {len(puntos_restantes)} puntos
restantes con k-NN (k={k_vecinos}) ---")

    for punto_nuevo in puntos_restantes:
        #Calculamos distancias
        distancias = [np.linalg.norm(punto_nuevo - p) for p in
puntos_a_clasificar]

        indices_vecinos = np.argsort(distancias)[:k_vecinos]

        etiquetas_vecinos = asignaciones[indices_vecinos]

        #Nos fijamos que grupo es mas comun en los vecinos
        prediccion = np.bincount(etiquetas_vecinos).argmax()
        predicciones_knn.append(prediccion)
        print(f"Punto {np.round(punto_nuevo, 2)} asignado al Grupo
{prediccion + 1}")

    #Grafico

```

```

plt.figure(figsize=(10, 8))
colores = ['blue', 'red']

for cluster_idx in range(k):
    puntos_del_cluster = puntos_a_clasificar[asignaciones ==
cluster_idx]
    plt.scatter(puntos_del_cluster[:, 0], puntos_del_cluster[:,
1],
                c=colores[cluster_idx], label=f'Grupo {cluster_idx
+ 1} (K-Means)', alpha=0.7)

    #Los nuevos 3 puntos
    colores_nuevos = [colores[p] for p in predicciones_knn]
    plt.scatter(puntos_restantes[:, 0], puntos_restantes[:, 1],
                c=colores_nuevos, marker='s', s=120,
                edgecolors='black', linewidth=1.5, label='Puntos
Clasificados (k-NN)')

    #Centroides finales
    plt.scatter(centroides[:, 0], centroides[:, 1], c='yellow',
marker='X', s=250,
                edgecolors='black', linewidth=1.5, label='Centroides
Finales')

    plt.title('Clasificación Híbrida: K-Means y k-NN')
    plt.xlabel('Coordenada X')
    plt.ylabel('Coordenada Y')
    plt.legend()
    plt.grid(True)
    plt.show()

ejecutar_kmeans_y_knn()

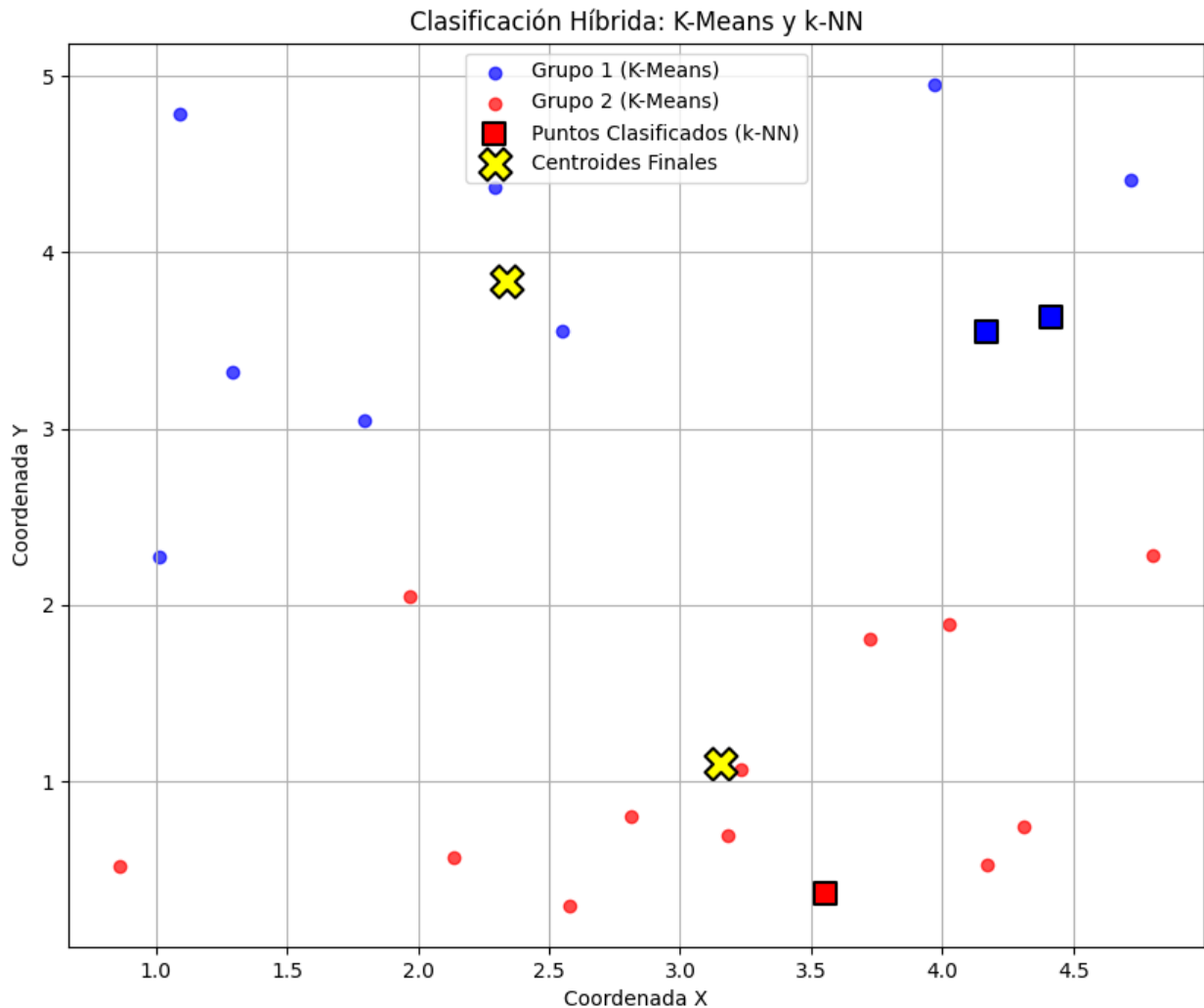
```

Algoritmo K-Means convergido en la iteración 2.

```

--- Clasificando 3 puntos restantes con k-NN (k=3) ---
Punto [3.56 0.36] asignado al Grupo 2
Punto [4.41 3.63] asignado al Grupo 1
Punto [4.17 3.55] asignado al Grupo 1

```

1. La imagen mostrada abajo muestra una red de salas y cómo se comunican entre ellas. Implemente un algoritmo Q-Learning con un factor despreciativo $\gamma = 0.9$. La matriz de recompensas debe asignar un valor de 0 a cada camino accesible y un valor de 100 a caminos que lleven a la sala del tesoro.

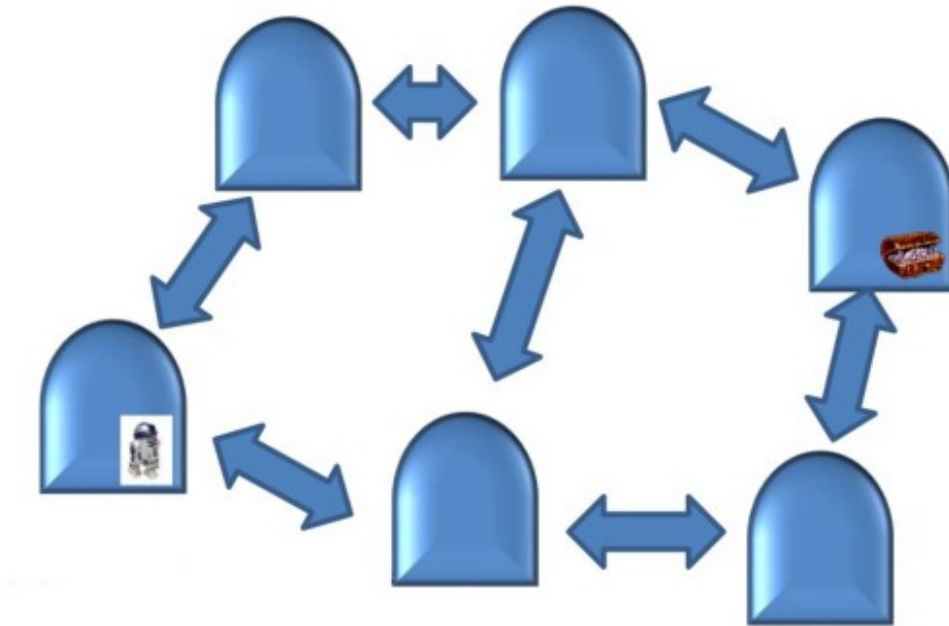
```
import requests
from PIL import Image
from io import BytesIO
import matplotlib.pyplot as plt

# URL directa de Google Drive
url = "https://drive.google.com/uc?export=view&id=1dkDruEIPa7f-BAmjI47TZl3bxaqYf6a9"

# Descargar la imagen
response = requests.get(url)
img = Image.open(BytesIO(response.content))

# Mostrar la imagen
```

```
plt.imshow(img)
plt.axis('off') # Ocultar ejes
plt.show()
```



4.1 Dibuje un diagrama con la asignación de recompensas correspondiente a cada estado.

4.2 Diseñe y muestre la matriz de recompensas.

4.3 Obtenga la matriz Q óptima, normalícela respecto al valor máximo encontrado y grafique la política obtenida en el diagrama mostrado inicialmente.

```
from PIL import Image
import matplotlib.pyplot as plt

# Abrir la imagen desde la carpeta del proyecto
img = Image.open("tp8_4_1.jpg")

# Mostrar la imagen con matplotlib
plt.figure(figsize=(img.width/80, img.height/80)) # Ajustá el divisor
para el tamaño
plt.imshow(img)
plt.axis("off")
plt.show()
```

MATRIZ DE RECOMPENSA						
	0	1	2	3	4	5
0	-1	0	0	-1	-1	-1
1	0	-1	-1	0	-1	-1
2	0	-1	-1	0	0	-1
3	-1	0	0	-1	-1	100
4	-1	-1	0	-1	-1	100
5	-1	-1	-1	0	0	-1

Tomando S=1

Seleccionando ir a 3 en Q

$$Q(1,3)=R(1,3)+0,9*\max[Q(3,1),Q(3,2),Q(3,5)]= \quad Q(1,3)=0+0,9*100=90$$

```
# Q-Learning para el grafo de 6 salas con tesoro en 5
# Reglas:
# - Conexiones bidireccionales: 0-1, 0-2, 1-3, 2-3, 2-4, 3-5, 4-5
# - Recompensas: 0 en transiciones válidas; 100 al entrar en 5
# - No hay auto-transiciones (i->i es imposible)
# - El episodio TERMINA al entrar a 5 (estado objetivo)
```

```
import numpy as np

def build_R(n_states=6, goal=5):
    """Construye la matriz de recompensas R con -1 en movimientos
    imposibles,
    0 en transiciones válidas y 100 al entrar a 'goal'. Sin auto-
    transiciones."""
    # Conexiones bidireccionales (pares no ordenados)
    undirected = [(0,1),(0,2),(1,3),(2,3),(2,4),(3,5),(4,5)]

    # Lista dirigida de aristas
    edges = []
    for (i,j) in undirected:
        edges.append((i,j))
        edges.append((j,i))

    R = -1 * np.ones((n_states, n_states), dtype=int)
    for (i,j) in edges:
        R[i, j] = 0
    # Entrar al goal da 100
    for s in range(n_states):
        if R[s, goal] != -1:
            R[s, goal] = 100
    # Sin auto-transiciones: mantengo la diagonal en -1 (incluye
    R[5,5] = -1)
```

```

    return R

def valid_actions(R, s):
    """Acciones válidas desde el estado s (columnas j donde R[s,j] !=
    -1)."""
    return np.where(R[s] != -1)[0]

def q_learning(R, gamma=0.9, alpha=0.8, episodes=6000, max_steps=30,
seed=7):
    """Ejecuta Q-Learning. El episodio TERMINA al entrar a goal.
    Devuelve la matriz Q aprendida."""
    n = R.shape[0]
    goal = np.argmax((R != -1).any(axis=0) & (R[:, :] ==
100).any(axis=0)) # asume único goal=5
    rng = np.random.default_rng(seed)
    Q = np.zeros_like(R, dtype=float)

    # Estados iniciales: cualquiera excepto el goal
    start_states = [s for s in range(n) if s != goal]

    for ep in range(episodes):
        s = rng.choice(start_states)
        for _ in range(max_steps):
            acts = valid_actions(R, s)
            a = rng.choice(acts) # política exploratoria simple
            (aleatoria)
            r = R[s, a]

            # Si entramos al goal, el episodio termina (sin valor
            futuro)
            if a == goal:
                target = float(r)
            else:
                next_acts = valid_actions(R, a)
                next_best = Q[a, next_acts].max() if next_acts.size
            else 0.0
                target = r + gamma * next_best

            # Actualización Q-Learning
            Q[s, a] = (1 - alpha) * Q[s, a] + alpha * target

            s = a
            if s == goal:
                break

    return Q

def derive_policy(R, Q):
    """Devuelve la mejor acción por estado (argmax sobre acciones
    válidas)."""

```

```

policy = {}
for s in range(R.shape[0]):
    acts = valid_actions(R, s)
    if acts.size:
        policy[s] = acts[np.argmax(Q[s, acts])]
return policy

def main():
    R = build_R(n_states=6, goal=5)
    Q = q_learning(R, gamma=0.9, alpha=0.8, episodes=6000,
max_steps=30, seed=7)
    policy = derive_policy(R, Q)

    # Mostrar matrices y política
    np.set_printoptions(precision=2, suppress=True)
    print("Matriz R (-1=imposible, 0=válida, 100=entrar a 5):")
    print(R)
    print("\nMatriz Q aprendida (aprox.):")
    print(Q)

    # Q normalizada (opcional, para lectura rápida)
    Q_max = Q.max() if Q.max() > 0 else 1.0
    print("\nQ normalizada (Q / Qmax):")
    print(Q / Q_max)

    print("\nPolítica (mejor acción por estado):")
    for s in sorted(policy.keys()):
        print(f"    desde {s} -> {policy[s]}")

```

```
main()
```

```
Matriz R (-1=imposible, 0=válida, 100=entrar a 5):
```

```

[[ -1   0   0  -1  -1  -1]
 [  0  -1  -1   0  -1  -1]
 [  0  -1  -1   0   0  -1]
 [ -1   0   0  -1  -1 100]
 [ -1  -1   0  -1  -1 100]
 [ -1  -1  -1   0   0  -1]]

```

```
Matriz Q aprendida (aprox.):
```

```

[[  0.  81.  81.   0.   0.   0. ]
 [ 72.9   0.   0.  90.   0.   0. ]
 [ 72.9   0.   0.  90.  90.   0. ]
 [  0.  81.  81.   0.   0. 100. ]
 [  0.   0.  81.   0.   0. 100. ]
 [  0.   0.   0.   0.   0.   0. ]]

```

```
Q normalizada (Q / Qmax):
```

```

[[0.   0.81 0.81 0.   0.   0. ]
 [0.73 0.   0.   0.9 0.   0. ]

```

```
[0.73 0.  0.  0.9 0.9 0. ]
[0.  0.81 0.81 0.  0.  1. ]
[0.  0.  0.81 0.  0.  1. ]
[0.  0.  0.  0.  0.  0. ]]
```

Política (mejor acción por estado):

```
desde 0 -> 1
desde 1 -> 3
desde 2 -> 3
desde 3 -> 5
desde 4 -> 5
desde 5 -> 3
```

```
from PIL import Image
import matplotlib.pyplot as plt

# Abrir la imagen desde la carpeta del proyecto
img = Image.open("tp8_4_2.jpg")

# Mostrar la imagen con matplotlib
plt.figure(figsize=(img.width/80, img.height/80)) # Ajustá el divisor
para el tamaño
plt.imshow(img)
plt.axis("off")
plt.show()
```

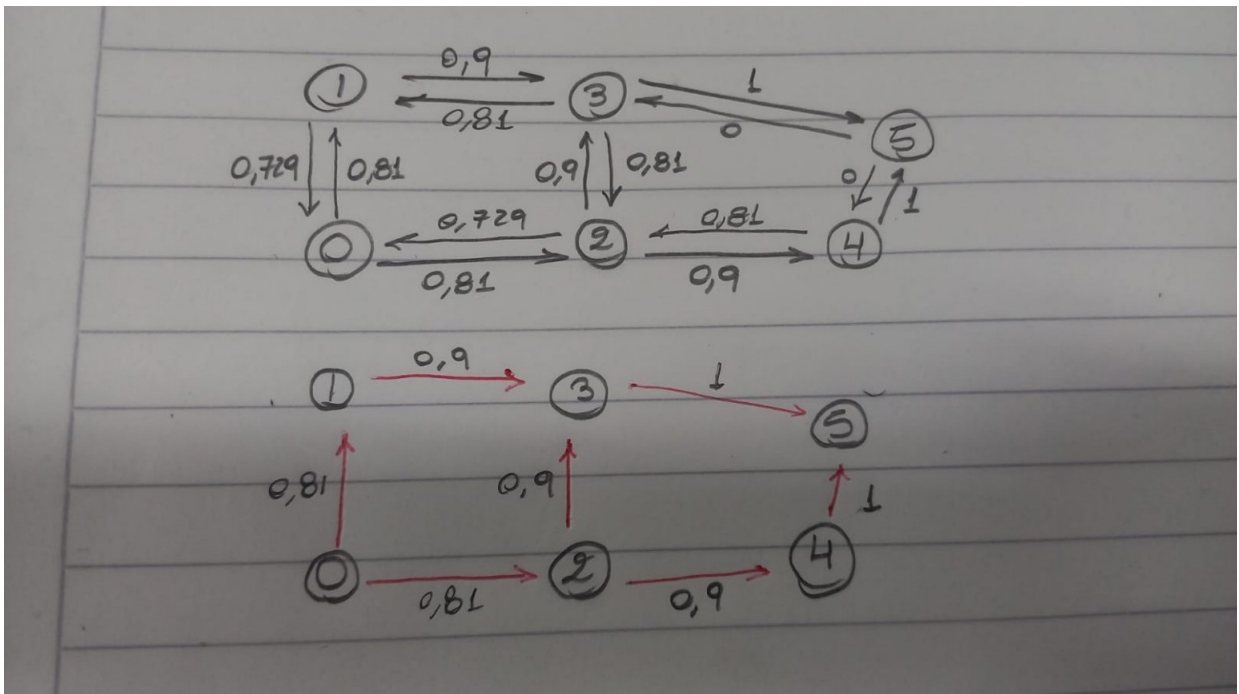


Diagrama de política obtenido

Bibliografía

Russell, S. & Norvig, P. (2004) *Inteligencia Artificial: Un Enfoque Moderno*. Pearson Educación S.A. (2a Ed.) Madrid, España

Poole, D. & Mackworth, A. (2017) *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press (3a Ed.) Vancouver, Canada